SYSTEM ANALYSIS DOCUMENTATION

(SAD)

DOMAIN 2

Project Name:

KeretaSewa AI: Decision Intelligence Platform for SME Car Rental Operators

Team Name: pakchoy

Introduction

KeretaSewa AI is a web based decision intelligence platform designed for small and medium sized car rental operators. The system helps operators understand fleet performance, identify underutilized vehicles, receive AI generated business recommendations, ask business questions in natural language, and generate dashboard reports.

1.1 Purpose

The purpose of this document is to describe how KeretaSewa AI is structured and how its main components interact with one another. This document covers the following areas:

- **System Architecture**
  KeretaSewa AI is implemented as a client side web application supported by an AI service layer. The frontend handles dashboard display, user interaction, data input, chart visualization, account settings, and PDF report generation. The AI service layer connects the application to OpenRouter API using Qwen3-32B model for recommendation generation and natural language decision support. Initially, GLM was tested during development, but the system was later switched due to temporary service instability.
- **Data Flow**
  The system receives vehicle and booking data through CSV upload or manual input. This data is processed into summary metrics such as revenue, utilization rate, idle days, booking count, and weekday versus weekend demand. These metrics are then displayed on the dashboard and passed to the AI service when recommendations or assistant responses are needed.
- **AI Model Process**
  The Qwen3-32B model through OpenRouter API is used as a reasoning layer. Instead of sending raw unstructured data blindly, the system provides selected fleet metrics and clear prompts. The model then returns structured recommendations or assistant responses that are displayed to the operator.

- **Technical Reference**
  This document serves as a reference for developers and testers. It explains the system boundaries, technology choices, expected behavior, risks, and future improvements.

1.2 Background

Many small and medium sized car rental operators manage their operations using simple tools such as spreadsheets, WhatsApp, manual records, or basic booking systems. These tools may be enough for recording bookings and communicating with customers, but they do not provide deeper analysis for business decision making.

KeretaSewa AI was created to address this gap. The system does not replace existing booking channels. Instead, it acts as an internal decision support layer for operators. It helps transform operational records into understandable dashboard metrics and AI supported business recommendations.

The current prototype focuses on proving the core concept: small rental operators can upload or enter basic fleet data, view business performance, receive AI recommendations, ask decision-related questions, and export dashboard reports.

1.3 Target Stakeholders

Stakeholders	**Roles**	**Expectations**
Rental Operator	Main user of the platform. Uploads fleet data, views dashboard analytics, asks AI questions, and reviews recommendations.	A simple system that helps identify problems and suggests practical business actions.
Business Owner / Manager	Uses insights and PDF reports for business planning and decision	Clear revenue, utilization, and recommendation

	making.	summaries that support planning.
Development Team	Builds and maintains frontend, data processing logic, AI service integration .	Modular codebase, clear data flow, stable API integration, and maintainable components.
QA / Testing Team	Tests user flows, dashboard behavior, AI responses, error handling, and report generation.	Defined functional scope, expected outputs, and edge case behavior.

System Architecture & Design

2.1 High-Level Architecture

2.1.1 Overview

Type	Details
System	Web Based Dashboard Platform
Architecture	Client Server Architecture

KeretaSewa AI is structured as a web based internal business platform where rental operators access the system through a browser dashboard. The frontend handles user interaction, navigation, charts, account management, and report generation.

The system processes uploaded fleet data locally and displays operational metrics such as revenue, bookings, utilization rate, idle days, and demand

patterns. When reasoning or decision support is required, the system sends structured business context to the OpenRouter API using Qwen3-32B model.

2.1.2 LLM as a Service Layer

OpenRouter API using Qwen3-32B model is integrated as a dedicated reasoning service rather than a generic AI component. The platform uses the model only for tasks that require analysis, recommendation generation, or natural language business assistance. Initially, the team tested GLM integration before switching due to temporary service instability.

The system uses the model in two major scenarios:

a. **AI Recommendation Generation**

Processed fleet metrics such as:

- revenue totals
- booking frequency
- idle vehicle days
- utilization rate
- weekday vs weekend demand
- underperforming vehicles

are packaged into a structured prompt and sent through OpenRouter API.

The model returns recommendation objects containing:

- suggested action
- reasoning
- expected impact
- confidence level
- Trade off analysis

b. Ask AI Assistant

When the user submits a natural language business question, the system combines:

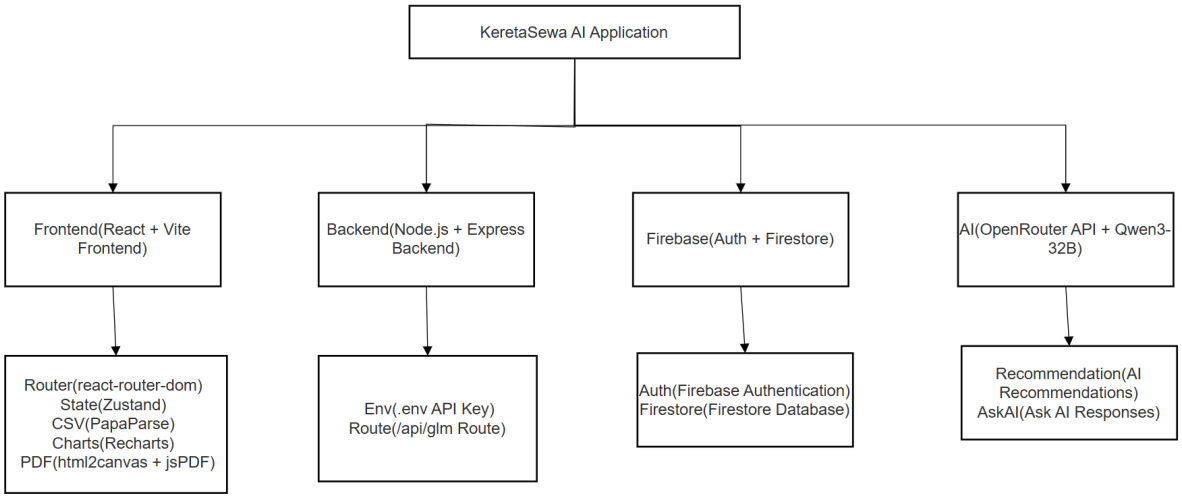
- user question
- current fleet summary
- business constraints

and sends them through OpenRouter API. The returned output is displayed as a structured business response.

Why It Is a Service Layer

The dashboard can still function without the model because core metrics are calculated by the system itself. However, recommendation generation and conversational decision support depend on this service layer.

2.1.3 Dependency Diagram



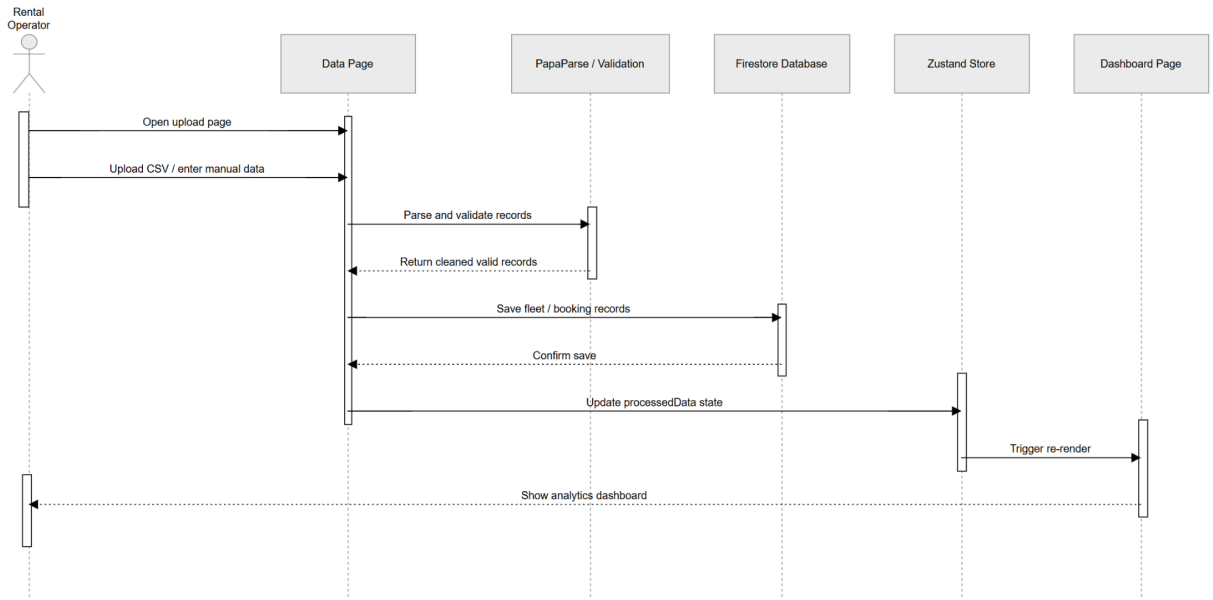
KeretaSewa AI uses a React frontend for user interaction, dashboard display, data upload, and report generation. Firebase Authentication manages user login, while Firestore stores user specific fleet and booking records. The frontend processes uploaded CSV data using PapaParse and stores shared application data through Zustand. For AI features, the frontend sends structured fleet context to a Node.js Express middleware server, which securely communicates with OpenRouter API using the Qwen3-32B model. The model response is returned to the frontend and displayed as recommendations or Ask AI answers.

Question	How KeretaSewa AI Addresses It
How prompts are constructed and sent to the model API	The frontend prepares fleet summary data and user questions, then sends them to the Node.js Express backend. The backend forwards the structured prompt to OpenRouter API using Qwen3-32B model.
What goes into the context window	Only summarized business metrics are included, such as revenue, utilization rate, idle days, total bookings, vehicle breakdown, and weekday versus weekend demand.
How responses are received, parsed, and passed to next component	The backend receives the model response and returns it to the frontend. The frontend parses the response into recommendation cards or Ask AI answers for display.
Where token limits are enforced / chunking before the model	Raw CSV data is not sent directly. The system summarizes fleet records into key metrics before sending them, reducing unnecessary token usage.
Major API calls between components	React frontend calls the Express backend through /api/glm. The backend then calls OpenRouter API. Firebase APIs are used separately for authentication and data storage.

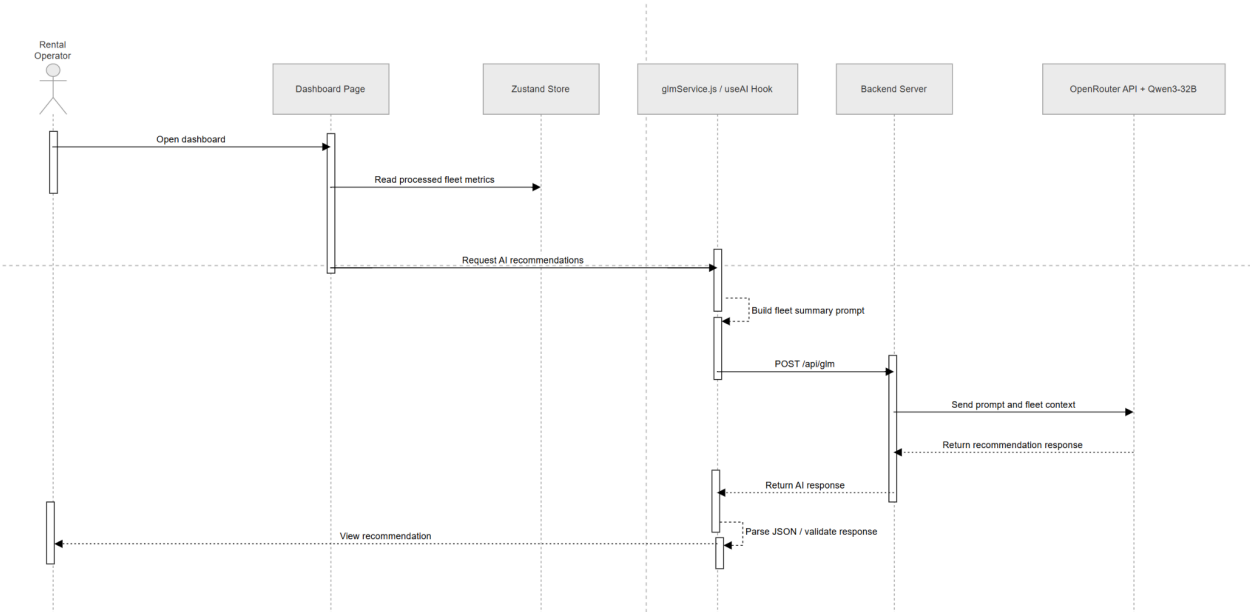
Frontend, backend, services, database, external API interaction	The frontend handles UI and user actions. Firebase stores user and fleet data. The Express backend securely connects to OpenRouter API. The model response is returned to the frontend for display.
Distributed components	The system has separate frontend, backend middleware, Firebase cloud services, and OpenRouter API. This separation allows the AI provider to be changed without redesigning the frontend.

2.2 Sequence Diagram

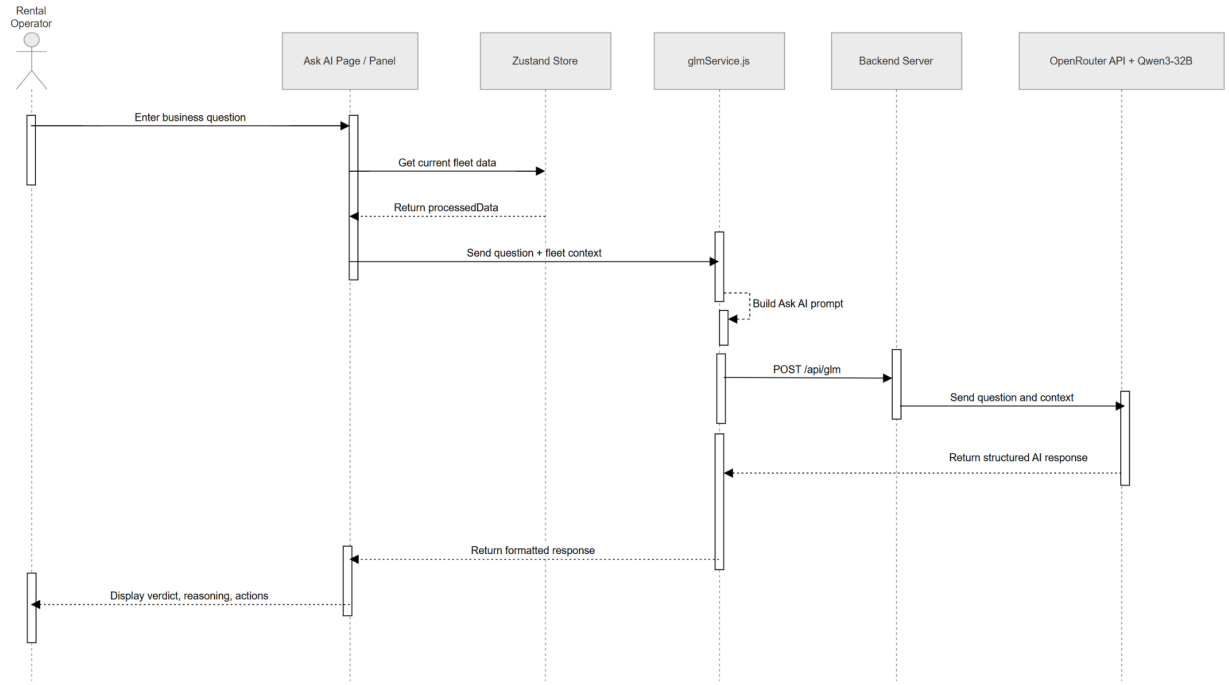
2.2.1 Upload Data and Display Dashboard Flow



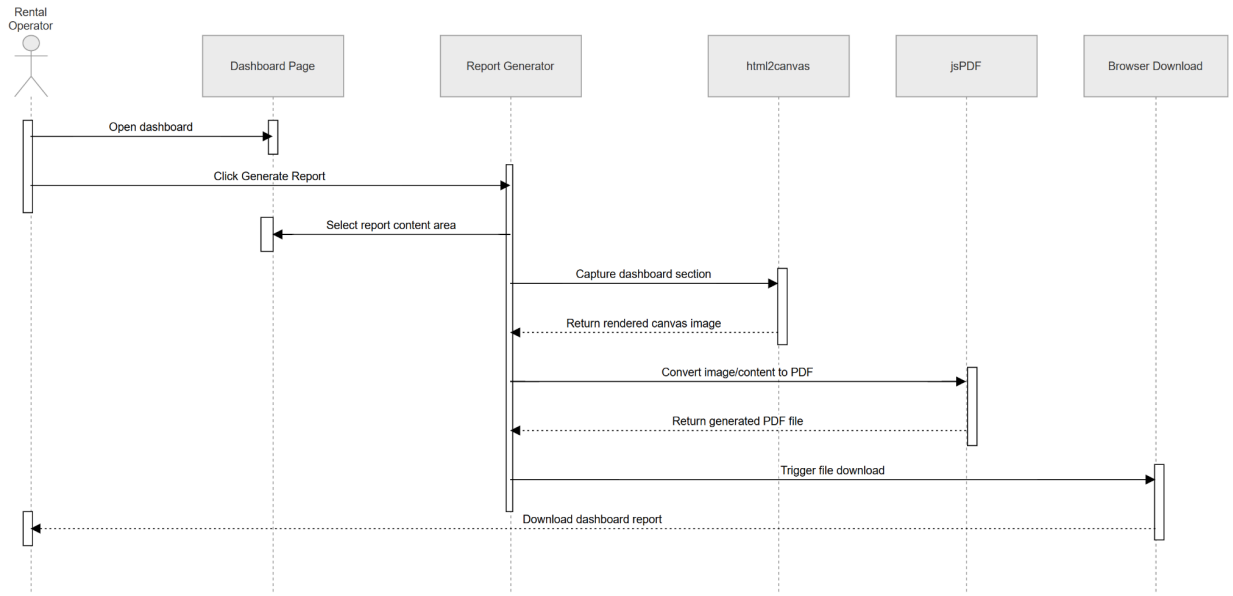
2.2.2 AI Recommendation Flow



2.2.3 Ask AI Assistant Flow



2.2.4 Generate PDF Report Flow



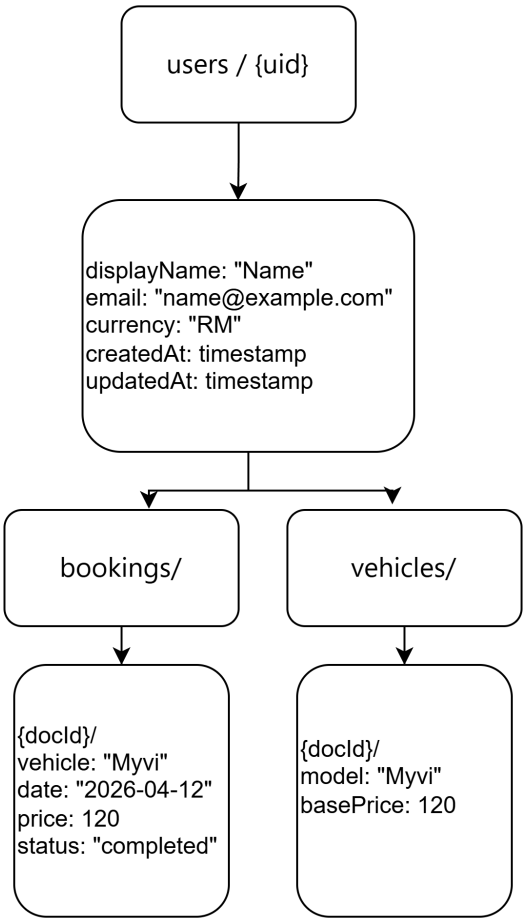
2.3 Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Builds a fast, responsive single-page web application for dashboard interaction, data upload, Ask AI, authentication pages, and report generation. Vite improves development speed and build performance.
Routing	react-router-dom	Handles page navigation between Login, Register, Dashboard, Data Upload, Ask AI, and Settings pages without full page reload.
State Management	Zustand	Manages shared frontend state such as uploaded fleet data, processed metrics, AI recommendations, chat history, loading states, and error handling across components.
Charts / Visualization	Recharts	Displays business analytics visually, including utilization charts, revenue metrics, and weekday versus weekend demand trends.
Backend API Layer	Node.js + Express	Provides the middleware server (server.js) that securely handles requests from the frontend, prepares prompts, and communicates with the OpenRouter API .
Database	Firebase Firestore	Stores uploaded fleet data, booking records, user profile data, and system-related information in a cloud-hosted NoSQL database.
Authentication	Firebase Authentication	Supports user registration, secure sign in, password updates, session handling, and account management.

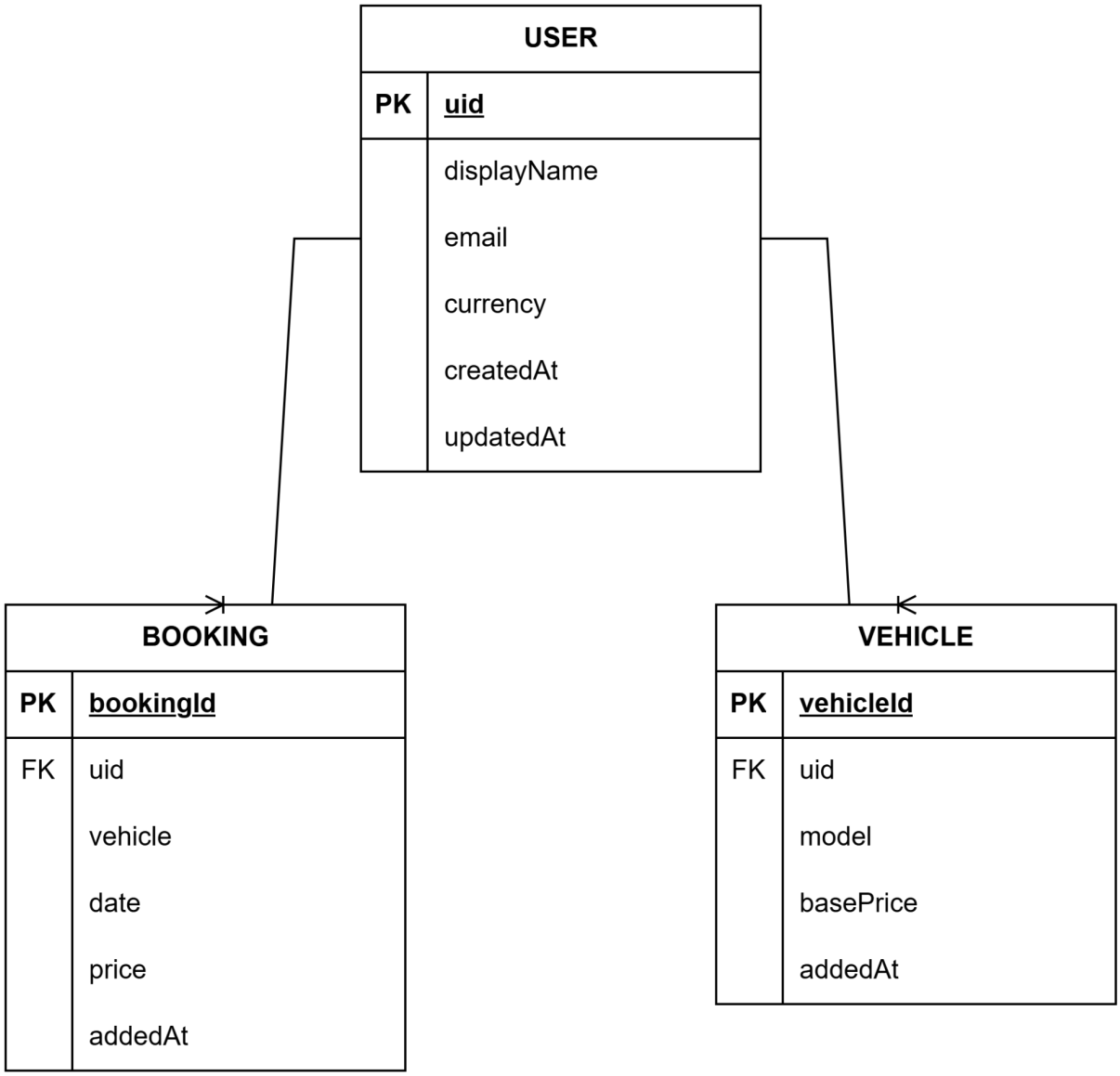
File Parsing	PapaParse	Parses uploaded CSV fleet files into structured JavaScript objects before processing and storage.
PDF Report Generation	html2canvas + jsPDF	Captures dashboard content and converts it into downloadable PDF business reports.
AI Service Layer	OpenRouter API using Qwen3-32B model	Generates AI recommendations and Ask AI responses using structured business prompts and fleet context. Initially, GLM was tested during development before switching due to temporary service instability.
Styling / UI	CSS / Tailwind / Custom Styling	Creates the modern dashboard interface, layout consistency, and responsive user experience.

2.4 Key Data Flows

2.4.1 Data Flow Diagram (DFD)



2.4.2 Logical Data Model



Functional Requirements & Scope

3.1 Minimum Viable Product (MVP)

Feature	Description
Data Input	Operators upload booking history via

	CSV or add/edit/delete individual records manually. Supports DD/MM/YYYY and YYYY-MM-DD date formats. New uploads append to existing data with no overwriting. Duplicate detection prevents double-counting.
Dashboard Analytics	Displays monthly fleet KPIs: total revenue, fleet utilization %, idle days, total bookings, weekday vs weekend demand split, and per-vehicle utilization bar chart. Month picker allows historical month browsing.
AI Recommendations Engine	OpenRouter API using Qwen3-32B model generates 3 context-aware recommendations per session with action, reasoning, expected impact, confidence score, trade-off analysis, and priority level. Recommendations are cached with timestamps and refreshed on demand.
Ask AI Assistant	Natural language Q&A interface where operators ask business questions. Qwen model receives fleet context with each question and returns structured responses with verdict, reasoning, and suggested action. Off-topic queries are redirected.
PDF Report Generation	Exports a formatted PDF containing fleet performance summary, metric cards, vehicle breakdown table, utilization visualization, weekday/weekend analysis, AI recommendations, and projected impact.
User Account Management	Firebase-authenticated accounts with profile editing (display name), password change with

	re-authentication, and permanent account deletion. Each user's data is fully isolated in Firestore.

3.2 Non-Functional Requirements (NFRs)

Quality	**Requirement**	**Implementation**
Security	API keys must never be exposed to the browser. User data must be isolated per account.	OpenRouter API key stored as server-side environment variable in Node.js middleware and .env file. Firestore security rules enforce request.auth.uid == userId on all read/write operations.
Reliability	Dashboard must remain functional if model API is unavailable. Failed AI calls must not crash the frontend.	OpenRouter API key stored as server-side environment variable in Node.js middleware and .env file. Firestore security rules enforce request.auth.uid == userId on all read/write operations.
Performance	Dashboard must render within 3 seconds after data load. Model API response must complete within 10 seconds for standard fleet sizes.	Metrics computed client-side in O(n) time. Firestore reads are parallelized using Promise.all(). Prompt capped at 1,500 tokens to minimize latency.
Token Cost Efficiency	Per-session token usage must remain within budget.	Prompt length fixed with structured schema. Fleet data summarized before injection. Repeated recommendation requests use cached results unless manually refreshed.
Usability	System must be usable by	Dashboard uses plain language

	non-technical car rental operators without training.	labels. AI recommendations are written in operator-friendly language. Validation messages are specific and actionable. Upload flow includes format guide and preview step.
Data Integrity	No duplicate booking records. All dates stored in consistent format.	Duplicate detection using vehicle + date composite key at upload. normalizeDate() converts all formats to YYYY-MM-DD before storage.
Scalability	System should support operators with up to 50 vehicles and 2 years of booking history.	Firestore scales horizontally. Metric computation filters by month before processing, only relevant records are loaded into memory. Token limit enforced for large datasets.

3.3 Out of Scope / Future Enhancements

- Reservation Management Module: the system does not handle new booking creation or customer-facing booking flows
- Payment Gateway Integration: no payment processing or invoice generation
- Live GPS Vehicle Tracking: no real-time vehicle location data
- Multi-Branch Enterprise Management: single operator account per session; no organizational hierarchy
- Automated Pricing Execution: recommendations are advisory only; the system does not automatically change prices on any booking platform
- Competitor Pricing Analysis: no external market data integration
- Demand Forecasting: predictive models for future booking volume are planned but not implemented
- Mobile Native Application: current build is browser-based desktop/laptop only
- Multi-Language Support: English only in the current prototype

- Real-Time Booking Integration: no live sync with existing booking systems or WhatsApp

Monitoring, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

4.1.1 Grayscale Rollout & A/B Testing

For future production deployment, KeretaSewa AI features should be released gradually instead of updating all users at once. This reduces operational risk and allows early issue detection.

Examples of features suitable for staged rollout:

- New dashboard card layout
- Improved AI recommendation prompt logic
- Faster CSV upload process
- Updated Ask AI chat interface
- New report export design

Rollout plan:

Phase	User Exposure	Objective
Phase 1	10% of users	Detect bugs, UI issues, slow loading, API failures
Phase 2	50% of users	Confirm stability under higher traffic

Phase 3	100% of users	Full release after successful validation

Metrics monitored during rollout:

- Dashboard loading time
- Recommendation success rate
- Ask AI response time
- Upload error rate
- User complaints / support feedback
- Browser compatibility issues

A/B Testing Strategy

A/B testing can be used when two versions of a feature are available.

Test Item	Version A	Version B
Recommendation Display	Compact cards	Detailed cards
Ask AI Response Style	Short summary	Full explanation
Dashboard Layout	4 metric cards	6 metric cards
Upload Flow	Single-step upload	Guided upload wizard

The version with better usability, engagement, and lower error rate can be selected.

4.1.2 Emergency Rollback (ER) & Golden Release

A Golden Release refers to the most recent stable version of KeretaSewa AI that has passed testing and can be restored immediately.

If a newly deployed version causes critical issues, Emergency Rollback should be triggered.

Trigger Conditions

Incident	Priority
Users cannot log in	P1
Dashboard page fails to load	P1
Backend server crashes	P1
Recommendations fail continuously	P2
CSV uploads corrupted	P2
PDF export broken	P3

Rollback Actions

1. Revert frontend deployment to previous stable build
2. Restore previous backend version
3. Temporarily disable unstable new feature

4. Notify users through maintenance banner
5. Investigate logs and redeploy fixed version later

Benefit

This minimizes downtime, prevents user frustration, and protects business continuity.

4.1.3 Priority Matrix

Priority	Trigger Condition	Action Taken	Response Target
P1 Critical	Login failure, dashboard inaccessible, backend offline	Immediate rollback and urgent patch	Within 15 minutes
P2 High	AI unavailable, upload failures, severe slowness	Hotfix or temporary feature disable	Within 1 hour
P3 Medium	PDF bug, chart display issue, partial UI bug	Fix in next patch cycle	Within 24 hours
P4 Low	Styling issue, wording issue, small layout inconsistency	Add to backlog	Next release

4.2 Monitoring Plan

KeretaSewa AI will monitor the main parts of the system to make sure users can still upload data, view dashboards, receive AI recommendations, and generate reports.

Area Monitored	What We Check	Trigger Condition	Action Taken
Frontend Dashboard	Page loading, UI errors, chart display	Dashboard cannot load or repeated frontend errors occur	Rollback latest frontend update or fix UI bug
Backend Server	/api/glm request success rate	Error rate above 5% within a short period	Check server logs, restart backend, apply hotfix
OpenRouter + Qwen Model	AI recommendation and Ask AI response success	Multiple failed AI requests or timeout above 10 seconds	Show user friendly error, allow retry, keep dashboard usable
Firebase Auth	Login and registration success	Users cannot sign in or register	Check Firebase status/rules and notify users if needed
Firestore Database	Read/write success	Fleet data cannot be saved or loaded	Check Firestore rules, database usage, and connection

CSV Upload	File parsing and validation result	High number of invalid upload errors	Review CSV validation and improve user guidance
PDF Report	Report generation success	PDF export fails repeatedly	Disable report button temporarily and patch issue

4.3 Assumptions

- Operators have stable internet connections when accessing the dashboard.
- Operators have basic digital literacy and can prepare a CSV file from their existing booking records.
- Booking records are maintained with at minimum vehicle name, date, and price information.
- OpenRouter API using Qwen3-32B model remains available and responsive during the hackathon demonstration.
- All operators use Malaysian Ringgit (RM) as their currency. Multi-currency support is considered a future enhancement.
- Operators currently manage a single business location. Multi-branch management is out of scope for the prototype.
- Date formats in uploaded CSV files follow either DD/MM/YYYY or YYYY-MM-DD conventions.
- Users access the system through modern web browsers such as Chrome, Edge, or similar supported browsers.

4.4 External Dependencies

Tool	Purpose	Risk Level	Mitigation
OpenRouter API using Qwen3-32B model	Core AI reasoning engine for recommendation generation and Ask AI responses	High. If unavailable, AI features fail entirely	Cache last successful recommendations in Firestore with timestamp. Display graceful error state. Manual refresh available.
Firebase Firestore	Persistent storage for all user data, booking records, vehicles, and cached AI output	High. If unavailable, data cannot be read or written	Firebase has 99.95% uptime SLA. Local Zustand state preserves session data if Firestore is temporarily unreachable.
Firebase Authentication	User identity and session management	High. If unavailable, no user can log in	Firebase Auth has high availability. No fallback, authentication is a hard dependency.
PapaParse	CSV parsing library	Low. Well-established library, no external API dependency	Bundled with the application. No runtime network dependency.
html2canvas + jsPDF	Client-side PDF generation	Low. Runs entirely in the browser	If generation fails, the dashboard remains fully functional. User shown retry option with error

			message.

Project Management & Team Contributions

5.1 Project Timeline

Phase	16 /4	17 /4	18 /4	19 /4	20 /4	21 /4	22 /4	23 /4	24 /4	25 /4	26 /4
Planning											
Core Data & UI											
Dashboard & Charts											
Auth & User Flow											
Data Management											
AI Integration											
Ask AI & Simulation											
Report Generation											
Testing											
Documentation											

5.2 Team Member Roles

Member	Role	Responsibilities
Cheong Yan Jie (Leader)	AI	OpenRouter API using Qwen3-32B model integration, prompt engineering, Node.js middleware server, JSON response parsing and validation, useAIRecommendations hook, Ask AI feature integration, business context injection, simulation logic, out-of-scope query redirect handling, prompt safety controls. Initially tested GLM integration during early development.
Tan Wei Ru	Frontend & Database	React component architecture, dashboard UI, CSV upload and parsing, metric computation, Firebase Firestore integration, Zustand state management, date normalization, DataPage (records + upload tabs), PDF report generation, authentication pages, landing page, settings page, routing and protected routes

5.3 Recommendations

Short-term (post-hackathon):

- Implement Redis caching for **OpenRouter API** responses to reduce repeated API calls and token usage for similar fleet contexts.
- Add response streaming for Ask AI requests to improve perceived response speed.
- Implement Jest unit tests for dataProcessor.js and projectionCalc.js to maintain metric accuracy as the codebase grows.
- Improve recommendation prompt templates for more consistent structured outputs.

Medium-term:

- Integrate competitor pricing data sources (pricing APIs or public market data) to provide better context for pricing recommendations.

- Add demand forecasting using historical booking records to estimate high and low demand periods.
- Build a mobile responsive layout for operators using tablets or smaller screens.
- Add multilingual support such as Mandarin and Bahasa Malaysia.

Long-term:

- Multi branch support with organizational hierarchy for operators managing multiple locations
- Direct integration with popular Malaysian booking platforms (e.g., Carsome, booking aggregators) for automated data sync
- Automated pricing execution which applies AI-recommended price changes directly to connected booking channels

Conclusion

KeretaSewa AI demonstrates that decision support tools are achievable for SME car rental operators without expensive enterprise systems. By combining a React-based dashboard, Firebase cloud services, a Node.js middleware layer, and OpenRouter API using Qwen3-32B model, the platform converts simple booking records into useful business insights.

The main value of the prototype is transforming raw CSV data into actionable recommendations. The system computes real utilization rates, revenue figures, idle day counts, and booking demand patterns from uploaded operator data. The integrated model then provides contextual recommendations and natural language assistance that would be difficult to achieve through fixed rule-based logic alone.

The architecture is intentionally modular. Core dashboard metrics continue to function independently of the model service, user data is isolated through Firestore security rules, and the recommendation layer is separated through the backend middleware server. This improves maintainability, reliability, and flexibility for future development.

Initially, GLM was integrated during development before switching to OpenRouter due to temporary service instability. This also demonstrates that the system can adapt to different model providers with minimal frontend changes.